Project 1: End-to-End Business Analytics

Python Analysis (EDA)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("../data/Superstore.csv", encoding="latin1")
df.head()
df.info()
df.isnull().sum()
```

```
Row ID           0
Order ID         0
Order Date       0
Ship Date        0
Ship Mode        0
Customer ID      0
Customer Name    0
Segment          0
Country          0
City             0
State            0
Postal Code      0
Region           0
Product ID       0
Category         0
Sub-Category     0
Product Name     0
Sales            0
Quantity         0
Discount         0
Profit           0
dtype: int64
```

```
df.info()
```

```
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   str
 2   Order Date     9994 non-null   str
 3   Ship Date      9994 non-null   str
 4   Ship Mode      9994 non-null   str
 5   Customer ID    9994 non-null   str
 6   Customer Name  9994 non-null   str
 7   Segment        9994 non-null   str
 8   Country        9994 non-null   str
 9   City           9994 non-null   str
 10  State          9994 non-null   str
 11  Postal Code    9994 non-null   int64
 12  Region         9994 non-null   str
 13  Product ID     9994 non-null   str
 14  Category       9994 non-null   str
 15  Sub-Category   9994 non-null   str
 16  Product Name   9994 non-null   str
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
dtypes: float64(3), int64(3), str(15)
```

```
df.columns
```

```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
       'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
       'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
       'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='str')
```

Fix Date Columns (DO THIS NOW)

```
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])
df.info()
```

```
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   str
 2   Order Date     9994 non-null   datetime64[us]
 3   Ship Date      9994 non-null   datetime64[us]
 4   Ship Mode      9994 non-null   str
 5   Customer ID    9994 non-null   str
 6   Customer Name  9994 non-null   str
 7   Segment        9994 non-null   str
 8   Country        9994 non-null   str
 9   City           9994 non-null   str
 10  State          9994 non-null   str
 11  Postal Code    9994 non-null   int64
 12  Region         9994 non-null   str
 13  Product ID     9994 non-null   str
 14  Category       9994 non-null   str
 15  Sub-Category   9994 non-null   str
 16  Product Name   9994 non-null   str
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
dtypes: datetime64[us](2), float64(3), int64(3), str(13)
```

Create New Columns (VERY IMPORTANT) *--For time base analysis*

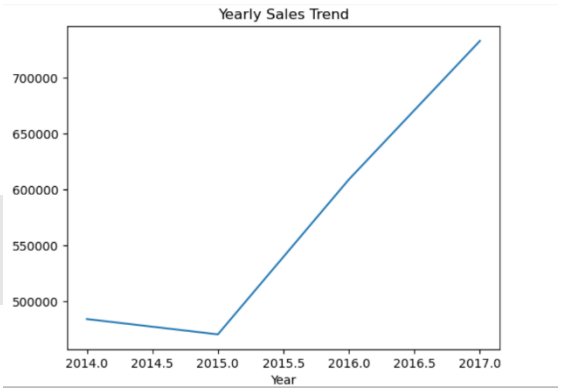
```
df['Year'] = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month
df['Day'] = df['Order Date'].dt.day
df.info()
```

```
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   str
 2   Order Date     9994 non-null   datetime64[us]
 3   Ship Date      9994 non-null   datetime64[us]
 4   Ship Mode      9994 non-null   str
 5   Customer ID    9994 non-null   str
 6   Customer Name  9994 non-null   str
 7   Segment        9994 non-null   str
 8   Country        9994 non-null   str
 9   City           9994 non-null   str
 10  State          9994 non-null   str
 11  Postal Code    9994 non-null   int64
 12  Region         9994 non-null   str
 13  Product ID     9994 non-null   str
 14  Category       9994 non-null   str
 15  Sub-Category   9994 non-null   str
 16  Product Name   9994 non-null   str
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
 21  Year           9994 non-null   int32
 22  Month          9994 non-null   int32
 23  Day            9994 non-null   int32
dtypes: datetime64[us](2), float64(3), int32(3), int64(3), str(13)
```

Key Analysis (IMPORTANT)

1. Revenue Trend

```
df.groupby('Year')['Sales'].sum().plot()
plt.title("Yearly Sales Trend")
plt.show()
```



WHY:

- To see business growth over time
- Helps answer:
  - Is company growing?
  - Which year performed best?

---

2. Top Categories

```
df.groupby('Category')['Sales'].sum().sort_values(ascending=False)
```

```
Category
Technology         836154.0330
Furniture          741999.7953
Office Supplies    719047.0320
Name: Sales, dtype: float64
```

WHY:

- Identify best-performing product categories

---

3. Profit Analysis

```
df.groupby('Category')['Profit'].sum()
```

```
Category
Furniture           18451.2728
Office Supplies    122490.8008
Technology         145454.9481
Name: Profit, dtype: float64
```

WHY:

- Sales ≠ Profit
- Some categories may sell more but earn less

---

4. Regional Sales

```
df.groupby('Region')['Sales'].sum()
```

```
Region
Central    501239.8908
East       678781.2400
South      391721.9050
West       725457.8245
Name: Sales, dtype: float64
```

WHY:

- Understand geographical performance

5. Customer Segmentation

```
df.groupby('Customer Name')['Sales'].sum().sort_values(ascending=False).head(10)
```

```
Customer Name
Sean Miller          25043.050
Tamara Chand         19052.218
Raymond Buch         15117.339
Tom Ashbrook         14595.620
Adrian Barton        14473.571
Ken Lonsdale         14175.229
Sanjit Chand         14142.334
Hunter Lopez         12873.298
Sanjit Engle         12209.438
Christopher Conant   12129.072
Name: Sales, dtype: float64
```

WHY:

- Identify top customers (VIP clients)

SQL Analysis

1. Create Database

```
CREATE DATABASE superstore_db;
USE superstore_db;
```

I first created a dedicated database to store and manage my project data.

2. Create Table (Based on Your Columns)

```
CREATE TABLE orders (
    Row_ID INT,
    Order_ID VARCHAR(50),
    Order_Date DATE,
    Ship_Date DATE,
    Ship_Mode VARCHAR(50),
    Customer_ID VARCHAR(50),
    Customer_Name VARCHAR(100),
    Segment VARCHAR(50),
    Country VARCHAR(50),
    City VARCHAR(50),
    State VARCHAR(50),
    Postal_Code VARCHAR(20),
    Region VARCHAR(50),
    Product_ID VARCHAR(50),
    Category VARCHAR(50),
    Sub_Category VARCHAR(50),
    Product_Name VARCHAR(255),
    Sales FLOAT,
    Quantity INT,
    Discount FLOAT,
    Profit FLOAT
);
```

Important:

- Removed spaces → replaced with **_**
- Used correct data types (DATE, FLOAT, INT)

3. Load the data

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Superstore.csv'
INTO TABLE orders
CHARACTER SET latin1
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(Row_ID, Order_ID, @Order_Date, @Ship_Date, Ship_Mode, Customer_ID, Customer_Name, Segment,
    Country, City, State, Postal_Code, Region, Product_ID, Category, Sub_Category, Product_Name, Sales,
    Quantity, Discount, Profit)
SET
Order_Date = STR_TO_DATE(@Order_Date, '%m/%d/%Y'),
Ship_Date = STR_TO_DATE(@Ship_Date, '%m/%d/%Y');
```

4. Queries:

```
-- Data Cleaning (Basic)
SELECT COUNT(*) FROM orders WHERE Sales IS NULL;
```

Output:

Count(*)
0

-- Total Sales

```
SELECT SUM(Sales) FROM orders;
```

WHY:

- SUM(Sales) → calculates total revenue
- Gives overall business performance

-- Sales by Category

```
SELECT Category, SUM(Sales) AS total_sales FROM orders GROUP BY Category ORDER BY total_sales
DESC;
```

WHY:

- GROUP BY splits data into categories
- Then calculates total sales per category

Output:

Category	Sales
Furniture	50000
Technology	80000

-- Top Customers

```
SELECT Customer_Name, SUM(Sales) as total_sales FROM orders GROUP BY Customer_Name ORDER
BY total_sales DESC LIMIT 10;
```

WHY:

- Group by customer
- Calculate their total spending
- Sort descending → highest first
- LIMIT 10 → top 10 customers

-- Regional Sales

```
SELECT Region, SUM(Sales) FROM orders GROUP BY Region;
```

WHY:

- Analyze performance by geography

*-- Profit by Category (**IMPORTANT**)*

```
SELECT Category, SUM(Profit) AS total_profit FROM orders GROUP BY Category ORDER BY
total_profit DESC;
```

-- Monthly Sales Trend

```
SELECT
    YEAR(Order_Date) AS year,
    MONTH(Order_Date) AS month,
    SUM(Sales) AS total_sales
FROM orders
GROUP BY year, month
ORDER BY year, month;
```

These SQL queries are doing the **same analysis you did in Python**, but directly on the database.

Python = analysis after loading data

SQL = analysis inside database

Power BI Project: Superstore Business Dashboard

Project Overview

This project focuses on building an interactive Power BI dashboard using the Superstore dataset to analyze sales performance, profitability, and customer insights.

Step 1: Data Loading

- Open Power BI Desktop
- Go to **Home → Get Data → Text/CSV**
- Select Superstore.csv
- Click **Load**

🛠 Step 2: Data Cleaning (Power Query)

- Click **Transform Data** to open Power Query Editor

  Fix Headers

  - Click **Use First Row as Headers**

  Fix Date Columns (Important)

  - Right click Order Date → Change Type → **Using Locale**
    - Data Type: Date
    - Locale: English (United States)
  - Repeat for Ship Date

  Fix Data Types

  - Sales → Decimal Number
  - Profit → Decimal Number
  - Quantity → Whole Number

  Apply Changes

  - Click **Close & Apply**

Step 3: Create DAX Measures

Go to **Modeling → New Measure**

Total Sales	Total Sales = SUM('Superstore'[Sales])
Total Profit	Total Profit = SUM('Superstore'[Profit])
Total Orders	Total Orders = COUNT('Superstore'[Order ID])

Step 4: Create Time Column

Month-Year Column	MonthYear = FORMAT('Superstore'[Order Date], "MMM YYYY")
Month Number (for sorting)	MonthNumber = MONTH('Superstore'[Order Date])

Sort MonthYear

- Select MonthYear column
- Click **Sort by Column → MonthNumber**

Step 5: Create Visualizations

KPI Cards

- Add Card Visual
- Drag:

    - Total Sales
    - Total Profit
    - Total Orders

Sales by Category (Bar Chart)

- Axis → Category
- Values → Total Sales

Sales by Region (Pie Chart)

- Legend → Region
- Values → Total Sales

Monthly Sales Trend (Line Chart)

- Axis → MonthYear
- Values → Total Sales

Step 6: Add Filters (Slicers)

Region Filter

- Add **Slicer**
- Field → Region

Category Filter

- Add Slicer
- Field → Category

Step 7: Dashboard Design

- Arrange visuals in structured layout:
    - KPI Cards on top
    - Charts in middle
    - Line chart below
    - Filters at bottom
- Remove duplicate visuals
- Align and resize charts properly

Key Insights

- Identified top-performing product categories
- Analyzed regional sales distribution
- Observed monthly sales trends and seasonality
- Identified high-value customers and revenue drivers

Conclusion

This dashboard provides an interactive view of business performance, enabling stakeholders to make data-driven decisions using real-time filtering and visualization.